

Team Engine Documentation

CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Adit Jigneshbhai Shah, Sai Sumedh Kaveti, Yeva Mheryan

Milestone 1 - Build Game Engine Foundations

Module 3 - Game Play Foundations

GitHub Repository: <https://github.com/CodeRockerr/CSC481-581-Group4>

(A) Introduction

In Milestone 1, we design the basic architecture of a reusable game engine which will make use of C++ and SDL3. The goal of the milestone is not about designing a commercial engine, but creating an engine with different modules so that the common modules will serve as the building blocks for the multiple games which the team will create. The engine will house functionalities like SDL initiation, window and renderer handling, entity definition, physics, keyboard handling, collision detection, and rendering scale handling. Specific game behaviors like player handling, enemy movement handling, game logic handling, and collision handling will not be in the engine. This makes it possible for every team member to independently create games based on the same engine framework. Common functionalities do not have to be repeated on every project, since improvements made on the common engine framework will improve all games at once.

(B) Design Goals

The development of the Milestone 1 engine was guided by the following design goals.

(a) Shared Engine Library

Each game is tied to a common library of the engine in the Engine/CMakeLists.txt file. Application-specific code for each game is kept in the Games/<name>/main.cpp file for each

teammate. This separation provides the basis for the distinction between generic engine features and application-specific game features.

(b) Generic Engine Interfaces

The engine operates on general concepts such as:

- position
- dimensions
- velocity
- textures
- rendering state
- behavioral flags

The engine lacks domain-specific abstractions such as "player," "enemy," and "platform." The role of the engine entities is defined by particular games. Thus, reusability is achieved through the possibility to use one and the same engine entity for different objects within a certain game.

(c) Separation of Collision Detection and Response

Collision detection takes place only in regards to deciding whether or not there is an overlap between two entities. The effects of the collision, such as stopping motion, doing damage or respawning a character are decided upon and implemented by each particular game.

(d) Cross-Platform Build Configuration

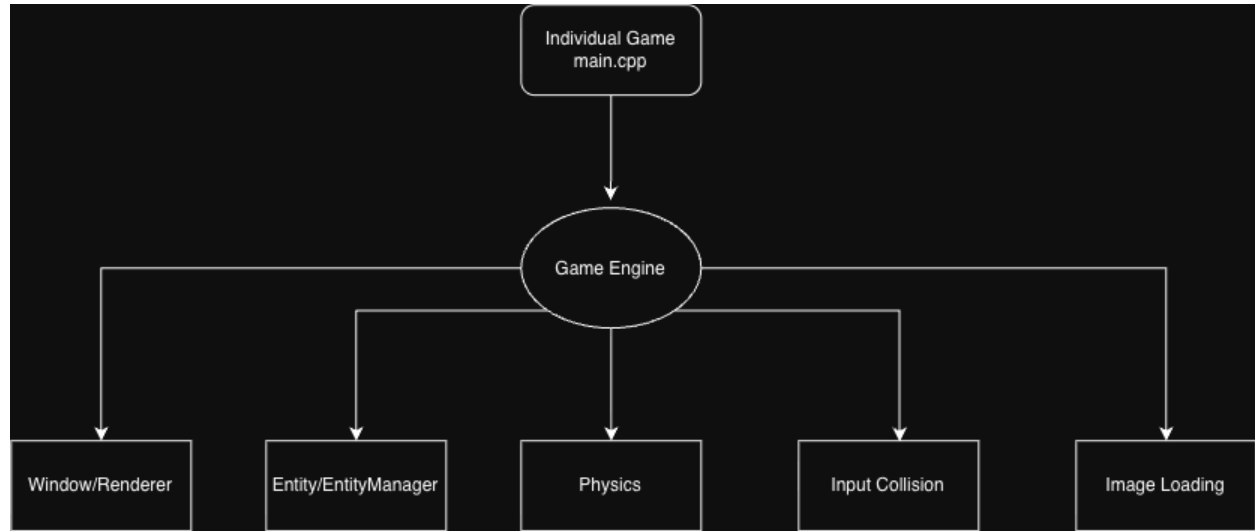
SDL3 has been included as a part of the repository in the vendored/SDL directory. Hence, there is no need for setting up a separate SDL version on the operating systems of choice. Thus, the same CMake project setup can be used on macOS and Windows.

(e) State-Based Keyboard Input

The processing of gameplay inputs occurs through `SDL_GetKeyboardState` instead of SDL key events. `SDL_PollEvent` will be used exclusively for OS level events like closing or resizing the application window. Gameplay inputs which occur continuously, like movement, are done based on the current keyboard state.

(C) Engine Architecture

The engine uses a modular structure where the Engine class is the main bridge between a specific game and the systems used by the engine.



(fig. 1 - Game Engine Structure)

The Engine is the facade that represents the subsystems. The constructor of the Engine initializes the SDL, initializes the application window and the renderer, and sets the reference resolution of the scaling system.

The primary application loop is implemented through:

C/C++

```
Engine::run(gameUpdate)
```

During each frame, the engine performs the following operations:

1. Handling window close and resize events.
2. Calculating frame deltaTime.
3. Updating keyboard state.
4. Executing physics calculations.
5. Running the per-frame callback for the game.
6. Updating positions of entities based on their velocities.
7. Clearing the frame buffer.
8. Rendering all entities.
9. Presenting the frame.

Games do not have a loop of their own to handle SDL events and rendering. Games create an Engine object, create entities, and provide a callback with specific game behavior.

Public engine interfaces are located in Engine/include/ while their implementations are located in Engine/src/. This company offers a traditional segregation of interface definitions and implementation details.

(D) Milestone Requirement Mapping

Task	Requirement	Primary Implementation
Task 1	SDL3 initialization, window, renderer, game loop, blue screen clear	Engine.cpp, Window.cpp, Renderer.cpp
Task 2	Generic drawable and updatable entity system	Entity.h, Entity.cpp, EntityManager.cpp
Task 3	Configurable gravity applied selectively to entities	Physics.h, Physics.cpp, Entity::affectedByGravity
Task 4	Keyboard-state polling through SDL_GetKeyboardState	Input.h, Input.cpp
Task 5	Boolean collision detection between two entities	Collision.h, Collision.cpp
Task 6	Pixel-based and percentage-based display scaling	EntityManager, ScaleMode, toggleScaleMode()

(E) Task 1 - Core Graphics and Engine Loop

(a) Implementation

SDL video functionality is initialized within the engine initialization process through:

```
C/C++  
SDL_Init(SDL_INIT_VIDEO)
```

Window is an abstract class for SDL window management. The default resolution that needs to be initialized is 1920×1080 and the window is configured to be resizable.

The abstract class Renderer handles the SDL renderer. Before drawing each frame, the renderer clears the screen using the color `RGBA(0, 90, 180, 255)` which creates the necessary blue background. After the rendering process is completed, `SDL_RenderPresent` presents the final frame. The complete frame lifecycle is coordinated by `Engine::run()`.

(b) Window Handling

Setting up a zero width and height allows auto-sizing according to about 80% of the usable area on the main monitor. The program is also designed to take into account monitors where a full application window of 1920×1080 would not fit comfortably. For this reason, the window sizes can be adjusted in order to allow use on smaller monitors. The process involves un-maximizing or un-zooming the application and setting up the requested size via SDL.

(c) Design Rationale

SDL initialization, rendering, and application loop processing being centralized ensures that there is no duplication of infrastructure in each game built by individual members of the team. Centralization of the SDL window and renderer objects ensures less dependency of SDL in the implementation of game play. The game play implementation is able to concentrate more on the game logic and not be concerned with graphics and resources.

(F) Task 2 - Generic Entity System

(a) Entity Representation

The Entity class provides a generic representation of an object that can exist within a game world.

Each entity stores information including:

- x and y position
- width and height
- horizontal and vertical velocity
- rendering information
- optional texture information
- state flags

Important state flags include:

```
C/C++
active
affectedByGravity
flipHorizontal
```

The entity can either be represented using a single color rectangle or the SDL texture. Since no gameplay functionality is attributed to entities by the engine, a single entity can be used to represent any game component such as a player, enemy, platform, background objects, projectiles, among others.

(b) Entity Manager

EntityManager manages the collection and lifecycle of entities.

Its primary operations include:

```
C/C++
createEntity(...)
setTexture(...)
updateAll(dt)
drawAll(...)
```

createEntity() allocates an entity and returns a pointer that can be retained by the game.

The manager stores entities internally using:

```
C/C++  
std::unique_ptr<Entity>
```

which provides centralized ownership and automatic memory cleanup.

updateAll(dt) performs position integration according to entity velocity “position = position + velocity × deltaTime”, while drawAll() renders each active entity using either its configured color or texture.

(c) Texture and Sprite Support

Entities may optionally use horizontal sprite strips.

For a texture containing multiple frames, the frame width is calculated as “frameWidth = textureWidth / frameCount”. The currently active frame may then be picked out through the corresponding source rectangle. The capability of horizontal flips is included as a rendering capability through the texture rotation feature of SDL along with SDL_FLIP_HORIZONTAL. This makes it possible for one sprite sheet to be used to portray entities moving in both directions.

(d) Design Rationale

A lightweight entity model was selected instead of creating a hierarchy such as:

```
C/C++  
Player : Entity  
Enemy  : Entity
```

If gameplay inheritance was used, then this would mean a loss of generality in the engine and changes would have to be made to the engine whenever a game added a new object type. In fact, the gameplay behavior is assembled externally using configuration of generic entities and modification of them through game code.

(G) Task 3 - Physics System

(a) Gravity Model

The Physics subsystem implements configurable gravitational acceleration.

The engine uses a default gravity value of 980.0 pixels/s² which can be modified at runtime through:

```
C/C++  
setGravity(float value)
```

and inspected using:

```
C/C++  
getGravity()
```

Gravity is not automatically applied to every entity. Instead, each entity contains the flag:

```
C/C++  
affectedByGravity
```

Only active entities with this flag enabled receive gravitational acceleration.

For each applicable entity $\rightarrow \text{velocityY} = \text{velocityY} + \text{gravity} \times \text{deltaTime}$

(b) Separation of Acceleration and Position Integration

Physics engine changes only velocity and not position. The task of position updating is left for `EntityManager::updateAll()`.

This produces a clean separation:

```
Shell  
Physics:  
    acceleration  $\rightarrow$  velocity  
  
EntityManager:
```

velocity → position

and correctly models gravity as an acceleration rather than a fixed displacement.

(c) Design Rationale

Gravity is not hardcoded but configured intentionally in different games. Different games are therefore able to configure different gravitational scenarios or even lack gravity.

Selective gravity also supports entities such as:

- backgrounds
- floating enemies
- moving platforms
- other objects that should remain unaffected by gravity

The present physics subsystem deliberately lacks elements like friction, rotation, or rigid body physics. Such elements are beyond the objective of Milestone 1 and can always be added in subsequent stages, if needed.

(H) Task 4 - Input Handling

(a) Keyboard State Management

Keyboard input is implemented through a static Input manager.

Its primary interface consists of:

C/C++

```
Input::update()  
Input::isKeyPressed(SDL_Scancode)  
Input::isKeyJustPressed(SDL_Scancode)
```

Input::update() is called once during each frame and reads the current SDL keyboard state through:

C/C++

```
SDL_GetKeyboardState()
```

The system holds information about the past state as well as the present state of the keyboard, allowing for continuous keystrokes to be recognized along with state changes.

(b) Continuous and Edge-Triggered Input

`isKeyPressed()` can be used for those actions that must be continued as long as the key is pressed down.

For example:

C/C++

```
if (Input::isKeyPressed(SDL_SCANCODE_W))
```

`isKeyJustPressed()` only checks for the change from not being pressed to being pressed.

This is useful for actions such as:

- jumping
- changing modes
- triggering menus
- toggling the Task 6 scaling mode

The edge detection method ensures that the pressed key does not cause the single action to occur again and again.

(c) Scancode-Based Input

It uses SDL scancodes instead of keycodes. The scancodes indicate physical positions on the keyboard and offer uniformity when it comes to popular control mechanisms like WASD.

(d) Separation from the Event Queue

`SDL_PollEvent` remains responsible for application and window events including:

C/C++

```
SDL_EVENT_QUIT
```

SDL_EVENT_WINDOW_RESIZED

The gameplay controller is not used through `SDL_EVENT_KEY_DOWN`. This ensures there is a clear distinction between events from the window system and gameplay input.

(I) Task 5 - Collision Detection

(a) Collision Model

Collision detection is exposed through:

```
C/C++  
Collision::checkCollision(  
    const Entity& a,  
    const Entity& b  
)
```

The method will return a Boolean which will indicate whether the two objects collide. Non-active objects will not be included in collision checking. The collision boundaries for each object will be provided by the `SDL_FRect` structure. Collision will be checked using SDL3's rectangle intersection method on floats. Thus, this system will use Axis-Aligned Bounding Box, also known as AABB, for collision detection.

(b) Detection Versus Response

The collision subsystem determines only whether a collision exists.

It does not automatically:

- move entities apart
- reverse velocity
- apply damage
- reset a player
- destroy entities
- trigger other gameplay behavior

For example:

```
C/C++  
if (Collision::checkCollision(*player, *ground)) {  
    // Game-specific response  
}
```

The game determines the appropriate response after receiving the collision result.

(c) Design Rationale

AABB collision is a cheap and general enough algorithm that will be useful for Milestone 1 games. It is better not to use a physics-based collision detection approach like collision manifolds or pixel-perfect collision detection at this point. The simple boolean interface gives the possibility to do different things when a collision occurs without changing anything in the common engine.

(J) Task 6 - Display Scaling System

(a) Scaling Modes

For the CSC 581 requirement, the engine supports two rendering modes:

```
C/C++  
ScaleMode::Pixel  
ScaleMode::Percentage
```

Pixel Mode

For entities in Pixel Mode, their dimensions remain unchanged despite resizing of the application window. For example, an entity configured as 100×100 pixels continues to render at that size after the window is resized.

Percentage Mode

For Percentage mode, the rendering parameters of the entity will be calculated based on the ratio between the current window size and the original reference resolution.

Conceptually:

$$\text{scaleX} = \text{currentWindowWidth} / \text{referenceWidth}$$
$$\text{scaleY} = \text{currentWindowHeight} / \text{referenceHeight}$$

The rendered values become:

$$\text{renderX} = \text{entityX} \times \text{scaleX}$$
$$\text{renderY} = \text{entityY} \times \text{scaleY}$$
$$\text{renderWidth} = \text{entityWidth} \times \text{scaleX}$$
$$\text{renderHeight} = \text{entityHeight} \times \text{scaleY}$$

Thus, the object will maintain roughly the same relative portion on the screen, regardless of how the window is resized.

(b) Runtime Toggle

The active scale mode can be changed using:

C/C++

```
EntityManager::toggleScaleMode()
```

Individual games can trigger this operation through the input system, for example:

C/C++

```
if (Input::isKeyJustPressed(SDL_SCANCODE_TAB)) {  
    entities.toggleScaleMode();  
}
```

`isKeyJustPressed()` guarantees that holding the key will not constantly alternate scaling modes every time there is a new frame.

(c) Design Rationale

Scaling is done during rendering and not through changing the dimensions of the entity itself.

It is significant because the dimensions of the entities can also be used for:

- collision detection
- physics
- positioning logic
- gameplay calculations

The alteration of the saved dimensions every time the window changes size could lead to the rendering and simulation state becoming inconsistent. With the engine performing the scaling of the rendered view alone, the simulation coordinate system remains consistent. The application dimensions at the start of the game are saved just once as the base resolution. Hence, the percentage scaling is always calculated against this resolution.

(K) Interaction Between Engine and Game Logic

A typical game uses the engine as follows:

```
C/C++
Engine engine("Game Title");

engine.getPhysics().setGravity(1600.0f);

Entity* ground = entities.createEntity(...);
Entity* player = entities.createEntity(...);
Entity* patrol = entities.createEntity(...);

player->affectedByGravity = true;

engine.run([&](float dt) {

    if (Input::isKeyPressed(SDL_SCANCODE_A)) {
        player->velocityX = -speed;
    }

    if (Input::isKeyJustPressed(SDL_SCANCODE_TAB)) {
        entities.toggleScaleMode();
    }

    if (Collision::checkCollision(*player, *ground)) {
        // Ground collision response
    }
}
```

```
    if (Collision::checkCollision(*player, *patrol)) {  
        // Game-specific response  
    }  
});
```

This example demonstrates the central architectural principle of the project that “**The engine provides reusable mechanisms, while each game defines gameplay policy**”.

It is responsible for handling all systems like rendering, physics, input state, and collision detection. The particular game will determine the entities present, player control schemes, behavior of the enemies, and what happens when entities collide.

(L) Build Configuration

The project uses CMake and a vendored copy of SDL3.

From the repository root, the project can be configured and compiled using

```
cmake -S . -B build  
cmake --build build
```

The shared engine is built as the engine library target.

Games are individually assembled to make individual executable targets residing in subdirectories under Games/. Games should be executed from the repository root when relative asset paths are used. The project is designed for C++17 or later with CMake 3.16 or later.

(M) Architectural Summary

Milestone 1 establishes six major capabilities within the shared engine:

1. Graphics-related core framework that is needed for SDL initialization, window handling, rendering and the application main loop.

2. Entity system which is capable of creating game entities either with texture or by their shape.
3. Physics system that supports gravitational acceleration.
4. Input system which uses states to process input actions and handles both continuous and edge input.
5. Separate AABB collision detection algorithm.
6. Rendering scale system for fixed-pixel and relative window scales.

Modularity and clear separation of roles have been made a point of emphasis in the implementation process. The engine systems allow for functionality that can be used over again but does not have any built-in assumption about the game being played. This design framework provides a suitable foundation upon which other milestones will be developed.

(N) External Libraries and Attribution

The project uses the following external components:

- SDL3 belongs in vendored/SDL, and it is responsible for platform-related features like window handling, rendering, key state handling, and rectangle intersection checks.
- stb_image is needed by the image loading functionality to process PNG images.

Both the common engine architecture and the specific source code of the game were developed by Group 4 for the course CSC 481/581 Game Engine Foundations, Fall 2026.

(O) Conclusion

The success of Milestone 1 is that it has been able to lay down the necessary architectural foundation that will allow the team to build the rest of its games. The project does not develop the typical SDL functionality separately for each individual game, but rather builds an engine that has all the common functionality in one library. The architecture allows for separating the engine from the actual game logic, defines the interfaces of its subsystems, and is flexible enough to be used in several team projects. Perhaps most importantly, Milestone 1 has provided the technical foundation on which future functionality could be built.